2025년도

학사학위논문

분산처리 기반 CCTV 화재 감지 시스템 개발: Kafka를 활용한 실시간 영상 분석

A Development of a Distributed Fire Detection System Using CCTV: Real-Time Video Analysis with Kafka

2025년 5월 26일

순천향대학교 공과대학
컴퓨터공학과

최승언

분산처리 기반 CCTV 화재 감지 시스템 개발: Kafka를 활용한 실시간 영상 분석

A Development of a Distributed Fire Detection System Using CCTV: Real-Time Video Analysis with Kafka

지도교수 하상호

이 논문을 공학사학위 논문으로 제출함

2025년 5월 26일

순천향대학교 공과대학
컴퓨터공학과

최승언

최 승 언 의 공학사학위논문을 인준함

2025년 5월 26일

심 사 위 원 이 해 각 인

지 도 교 수 하 상 호 인

순천향대학교 공과대학

컴퓨터공학과

초 록

본 연구는 CCTV 기반 실시간 화재 감지 시스템의 성능과 확장성을 향상시키기 위한 분산 처리 인프라의 설계에 관한 것이다. 현대의 영상 분석 시스템은 다양한 환경에서 다수의 영상 스트리밍 데이터를 동시에 처리해야 하며, 이 과정에서 처리 지연, 병목 현상, 단일 장애점(SPOF; Single Point of Failure) 등의 문제가 발생할 수 있다.

특히 기존 화재 감지 시스템은 주로 센서 기반 또는 단일 서버 중심의 구조로 구성되어 있어, 다수의 CCTV로부터 유입되는 실시간 영상 데이터를 안정적으로 처리하기에는 구조적인 한계를 가진다. 이에 본 논문에서는 Apache Kafka를 중심으로 하는 메시지 기반 비동기 분산 처리 구조를 설계하고, 프레임 단위 영상 분석을 통해 객체 단위의 화재 징후를 탐지하는 시스템 아키텍처를 제안한다.

각 CCTV로부터 수신되는 RTSP 스트리밍 영상은 OpenCV를 활용하여 프레임으로 분할되고, Kafka Producer를 통해 메시지 큐로 전송된다. Kafka는 수신된 메시지를 Partition 단위로 분산 저장하며, 분석 서버는 Kafka Consumer Group으로 구성되어 PyTorch 기반의 YOLOv5 모델을 통해 연기 및 화염 객체를 병렬로 탐지한다. YOLOv5는 경량화된 구조와 높은 처리 속도를 갖춘 실시간 객체 탐지 모델로, 화재 상황과 같이 빠른 대응이 요구되는 환경에 적합하다. 분석 결과는 로컬 파일 또는 데이터베이스에 저장되며, 처리 과정은 프레임 단위로 독립적으로 수행되어 시스템의 지연을 최소화하고 병목 현상을 줄일 수 있다. 또한 분석 서버의 수와 Kafka 구성은 시스템 요구에 따라 유연하게 조정 가능하여, 다양한 환경에서의 처리량 변화에 능동적으로 대응할 수 있다.

제안된 시스템은 실시간 데이터를 지속적으로 수용할 수 있는 처리 구조를 바탕으로, 실제 감시 환경에서도 적용 가능한 인공지능 기반 화재 감지 인프라로 기능할 수 있다.

키워드: Kafka, 분산 처리 구조, PyTorch, YOLOv5, 객체 탐지, RTSP, OpenCV,

ABSTRACT

This study focuses on the design of a distributed processing infrastructure aimed at enhancing the performance and scalability of real-time fire detection systems based on CCTV video streams. Modern video analysis systems are required to handle multiple streaming data sources simultaneously across diverse environments. During this process, challenges such as processing delays, bottlenecks, and single points of failure (SPOF) often arise.

Conventional fire detection systems are typically based on sensor-driven or single-server architectures, which pose structural limitations in reliably processing real-time video data from numerous CCTV sources. To address this, the proposed study presents an asynchronous message-based distributed processing architecture centered on Apache Kafka. The system performs frame-level video analysis to detect fire-related objects, such as smoke and flames.

RTSP video streams from each CCTV camera are segmented into individual frames using OpenCV and transmitted via a Kafka Producer into a message queue. Kafka distributes the incoming messages into partitions, while the analysis servers, organized as a Kafka Consumer Group, utilize a PyTorch-based YOLOv5 model to perform parallel object detection. YOLOv5 is a lightweight and fast real-time object detection algorithm, making it suitable for environments that require rapid response, such as fire emergencies.

Detection results are stored either locally or in a database, and each frame is processed independently to minimize system latency and reduce bottlenecks. The number of analysis servers and the Kafka configuration can be flexibly scaled according to system requirements, enabling the system to respond dynamically to varying workloads in different environments.

The proposed system, built on a real-time data ingestion and processing framework, demonstrates the potential to serve as an AI-based fire detection infrastructure applicable to actual surveillance scenarios.

Keywords: Kafka, Distributed Processing Architecture, PyTorch, YOLOv5, Object Detection, RTSP, OpenCV

차 례

제 1 장 서 론 ..... - 1 -

제 2 장 관련 기술 ..... - 3 -

2.1 화재 감지 기술 개요 ..... - 3 -

2.2 객체 탐지 기술 ..... - 4 -

2.2.1 YOLO(You Only Look Once) ..... - 5 -

2.2.2 YOLO 의 내부 구성 요소 ..... - 6 -

2.2.3 YOLOv5 의 특징 ..... - 7 -

2.2.4 전처리 및 후처리 기법 ..... - 7 -

2.3 Kafka 메시징 시스템 ..... - 9 -

2.4 RTSP 스트리밍과 OpenCV 기반 영상 처리 ..... - 11 -

2.5 FastAPI 기반 결과 수신 시스템 ..... - 12 -

제 3 장 시스템 설계 및 구현 ..... - 14 -

3.1 YOLOv5 기반 객체 탐지 모델의 설계 및 구현 ..... - 16 -

3.1.1 학습 데이터 수집 ..... - 16 -

3.1.2 학습 데이터 전처리 및 가공 ..... - 16 -

3.1.3 모델 학습 환경 및 설정 ..... - 18 -

3.1.4 모델 저장 및 추론 활용 방식 ..... - 18 -

3.2 Kafka 기반 분산 시스템 설계 및 데이터 흐름 ..... - 19 -

3.2.1 Kafka 구성 요소 및 역할 ..... - 20 -

3.2.2 토픽 구조 설계 ..... - 21 -

3.2.3 메시지 처리 흐름 - 22 -
3.3 FastAPI 서버 및 Android 애플리케이션 설계 - 23 -
3.3.1 FastAPI 서버 구성 및 역할 - 24 -
3.3.2 Android 애플리케이션 구성 및 기능 - 25 -
제 4 장 실행 및 결과 분석 - 27 -
4.1 성능 평가 기준 - 27 -
4.2 객체 탐지 정확도 평가 - 28 -
4.3 Kafka 기반 시스템 처리 흐름 테스트 - 31 -
4.3.1 처리 흐름 요약 - 31 -
4.3.2 실험 방법 및 측정 기준 - 31 -
4.3.3 단일 Consumer 환경 결과 - 32 -
4.3.4 병렬 Consumer 환경 결과 - 32 -
4.3.5 성능 개선 정리 - 33 -
4.5 Android 애플리케이션 동작 테스트 - 33 -
4.5.1 감지 알림 수신 테스트 - 34 -
4.5.2 감지 목록 조회 및 이미지 렌더링 테스트 - 35 -
5. 결론 및 향후 과제 - 37 -
참 고 문 헌 I
감사의 글 III

그 림 차 례

[그림 1] YOLOv4 아키텍처 다이어그램 - 5 -

[그림 2] Letterbox 전처리 과정 - 8 -

[그림 3] Kafka 구조 - 11 -

[그림 4] 시스템 설계 - 15 -

[그림 5] Kafka 기반 화재 감지 시스템의 메시지 처리 흐름 - 20 -

[그림 6] FastAPI 서버와 Android 애플리케이션 간의 데이터 흐름 - 24 -

[그림 7] Android 애플리케이션 내 감지 결과 목록 UI 구성 - 26 -

[그림 8] YOLOv5 검증 결과 요약 (터미널 출력) - 28 -

[그림 9] Confidence 값에 따른 F1 점수 변화 - 29 -

[그림 10] 클래스별 Precision–Recall 곡선 - 30 -

[그림 11] 알림 기능 테스트 - 34 -

[그림 12] 목록 조회 테스트 - 35 -

표 차 례

[표 1] 데이터 구분 ..... - 17 -

[표 2] 성능 지표 ..... - 28 -

[표 3] 단일 Consumer 환경 결과 ..... - 32 -

[표 4] 병렬 Consumer 환경 결과 ..... - 33 -

[표 5] 성능 개선 정리 ..... - 33 -

제 1 장 서 론

현대 사회에서 화재는 산업 현장, 공장, 주거지 등 다양한 공간에서 발생하며, 생명과 재산에 큰 피해를 초래하는 재난이다. 특히 무인 공간에서는 조기 감지와 신속한 대응이 필수적이나, 현재의 자동화재감지 시스템은 오작동과 같은 문제로 실효성이 낮은 경우가 많다.[1]

실제로, 2024 년 상반기 충남 지역에서 자동화재속보설비를 통한 화재 신고 2546 건 중 실제 화재는 단 11 건(0.4%)에 불과했으며, 대부분이 감지기 오작동으로 인한 허위 신고였다. 고온, 습기, 먼지 등 다양한 환경 요인이 오작동의 원인으로 지적되고 있으며, 이로 인해 오작동을 이유로 건물 관리자들이 감지기나 스프링클러를 아예 꺼버리는 등 초기 대응 시스템이 무력화되는 악순환과 계속되는 오작동 허위 신고로 인한 소방 인력의 불필요한 출동과 초기 대응 지연 문제가 지속되고 있다.[2]

이와 같은 한계 속에서 최근 주목받고 있는 기술은 영상 기반 인공지능 감지 시스템이다. 특히 공공 CCTV 인프라의 급속한 확장은 이러한 기술 적용의 기반을 더욱 강화하고 있다. 2023 년 기준 서울시에는 약 14 만 5000 대의 CCTV 가 설치되어 있으며, 이는 단위 면적당 세계 2 위에 해당하는 밀도로, 2021 년보다 약 두 배 증가한 수치이다.[3] 대도시를 중심으로 영상 감시 환경이 빠르게 고도화됨에 따라, 기존 감지기와는 다른 방식의 화재 감지 시스템이 현실적으로 구현 가능한 기반이 마련된 것임과 동시에 실시간 감시 데이터를 기반으로 한 효율적인 분석 시스템의 필요성이 더욱 부각되고 있는 상황이다.

이에 본 논문은 대용량 데이터 스트리밍을 처리하기 위한 고성능 분산 메시징 플랫폼인 Apache Kafka 와 실시간 객체 탐지 알고리즘인 YOLOv5 를 활용하여 실시간 CCTV 영상 기반의 화재 감지 인프라를 설계하고 구현하는 것을 목표로 한다. 제안된 시스템은 실시간 영상 스트리밍, 객체 탐지, 결과 저장 및 모바일 알림 전송까지의 흐름을 포함하며, 분산 아키텍처 기반의 확장성과 실시간성을

동시에 확보하고자 한다. 논문의 구성은 다음과 같다. 2 장에서는 본 연구와 관련된 기술적 배경을 살펴보고, 3 장에서는 전체 시스템의 설계 및 구현 과정과 구성 요소별 세부 구현 내용을 기술하며, 4 장에서는 시스템의 성능을 실험을 통해 평가한다. 마지막으로 5 장에서는 본 연구의 결론을 정리하고, 향후 보완할 점과 발전 방향에 대해 제안한다.

제 2 장 관련 기술

다수의 CCTV 로부터 실시간으로 수신되는 영상 데이터를 분석하여 화재를 감지하기 위해서는, 단순히 객체 탐지 모델만 도입하는 것만으로는 부족하다. 화재는 초기 대응의 속도가 피해 규모를 결정짓는 중요한 요소이기 때문에, 실시간성과 높은 정확도를 동시에 만족시키는 기술적 기반이 필요하다. 특히 다수의 CCTV 로부터 영상이 동시에 수신되는 상황에서는 분석 처리의 병렬성과 시스템의 확장성이 핵심적인 고려 대상이 된다.

본 논문에서는 YOLOv5 객체 탐지 모델을 중심으로, Kafka 기반의 메시지 스트리밍 구조를 통해 프레임 전송과 분석을 분산 처리할 수 있는 시스템을 설계하였다. RTSP 프로토콜을 통해 CCTV 스트림을 수신하고, OpenCV 를 이용해 프레임을 추출하며, Kafka 를 통해 이를 전달 및 수신하는 구조를 통해 실시간 영상 분석 흐름을 구성하였다. 분석된 결과는 데이터베이스에 저장되고, FastAPI 와 Firebase Cloud Messaging(FCM)을 통해 관리자에게 실시간 알림 및 감지 목록 조회 기능을 제공한다.

이 장에서는 시스템을 구성하는 핵심 기술 요소들을 소개하고, 각 기술이 수행하는 역할과 시스템 내에서의 연계 방식을 설명한다. 구체적으로는 화재 감지 기술의 개요, YOLO 객체 탐지 알고리즘, Kafka 메시징 시스템, RTSP 기반 영상 처리, FastAPI 서버 및 알림 기술을 중심으로 서술한다.

2.1 화재 감지 기술 개요

화재 감지 기술은 일반적으로 열 감지 센서, 연기 감지 센서, 불꽃 센서 등을 기반으로 발전해왔다. 이러한 센서 기반 시스템은 반응 속도가 빠르고 구조가 간단하다는 장점이 있으나, 설치 환경의 제약, 좁은 감지 범위, 오탐지 발생 등 여러 한계가 존재한다.[1]

최근에는 영상 기반 화재 감지 기술이 주목받고 있으며, 이는 CCTV 카메라 등으로부터 수집된 영상을 인공지능 모델로 분석하여 화재를 탐지하는 방식이다. 이 접근은 넓은 범위를 커버할 수 있고, 시각적 증거 확보가 가능하며, 인공지능의 발전과 함께 정확도와 실시간성 측면에서도 빠르게 성능이 개선되고 있다.[4]

2.2 객체 탐지 기술

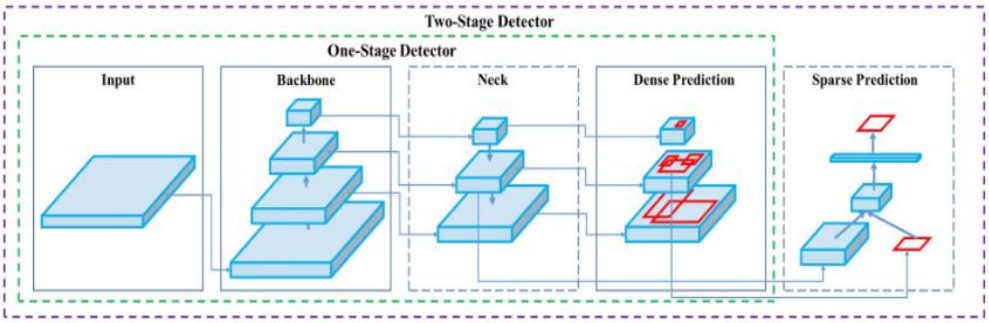
객체 탐지(Object Detection)는 이미지 또는 영상 내에 존재하는 객체의 위치와 종류를 동시에 예측하는 컴퓨터 비전 기술이다. 객체 탐지는 입력된 시각 데이터를 분석하여 객체의 존재 여부를 판단하고, 바운딩 박스를 통해 공간적인 위치를 지정하는 방식으로 작동한다. 이 기술은 실시간 영상 처리, 자율 주행, 보안 감시, 화재 감지 등 다양한 분야에서 활용된다.[5]

객체 탐지 알고리즘은 구조적 차이에 따라 Two-Stage Detector 와 One-Stage Detector 로 구분된다. Two-Stage 방식은 먼저 객체가 있을 법한 영역을 후보로 설정하고 그 영역을 분류하는 단계로 구성되며, 정확도는 높지만 속도는 느리다. 반면 One-Stage 방식은 입력 이미지를 한 번에 처리하여 객체를 직접 탐지하는 구조로, 빠른 속도가 장점이다. 대표적인 One-Stage 알고리즘에는 YOLO(You Only Look Once)와 SSD(Single Shot MultiBox Detector) 등이 있다.

YOLO(You Only Look Once)는 One-Stage 구조를 기반으로 하는 대표적인 객체 탐지 알고리즘으로, 전체 이미지를 일정한 크기의 그리드로 분할한 뒤 각 셀에서 객체의 위치와 클래스 정보를 동시에 예측한다. 이러한 구조는 높은 처리 속도와 실시간 분석에 적합한 특징을 지니며, 다음의 세 가지 주요 구성 요소로 이루어진다[6].

먼저, Backbone 은 합성곱 신경망(CNN)을 기반으로 한 특징 추출기로서, 입력 이미지로부터 의미 있는 시각 정보를 추출하는 역할을 수행한다. 이어서, Neck 은 서로 다른 해상도의 특징 맵을 결합함으로써 다양한 크기의 객체를 효과적으로 탐지할 수 있도록 중간 표현을 강화하는 계층이다. 마지막으로,

Dense Prediction 단계에서는 각 셀 위치에 대해 객체 존재 여부, 경계 상자 좌표, 그리고 클래스 확률을 직접 출력함으로써 최종 탐지 결과를 도출한다.[6]



[그림 1] YOLOv4 아키텍처 다이어그램

2.2.1 YOLO(You Only Look Once)

YOLO는 객체 탐지를 위한 단일 단계(one-stage) 알고리즘 중 하나로, 하나의 신경망 모델이 이미지 전체를 한 번에 입력 받아 객체의 위치와 클래스를 동시에 예측하는 구조를 갖는다. 기존의 두 단계 방식(two-stage)보다 처리 속도가 훨씬 빠르다는 장점이 있으며, 실시간 영상 처리에 특히 적합하다.

YOLO 는 입력 이미지를 $S \times S$ 크기의 그리드로 분할하고, 각 셀에서 B 개의 바운딩 박스를 예측한다. 각 바운딩 박스는 중심 좌표 (x, y), 폭과 높이 (w, h), 객체 존재 확률(objectness score), 그리고 클래스 확률 분포(class probabilities)를 포함하는 값을 출력한다[6].

이러한 예측 결과는 다음과 같은 형태의 출력을 구성하며, 여기서 CCC 는 탐지 대상 객체 클래스의 수를 나타낸다.

$$\text{Output Tensor Shape} = S \times S \times (B \times (5 + C))$$

이 수식에서 괄호 안의 5 는 각각의 바운딩 박스에 대해 위치 4 개 (x, y, w, h)와 객체 존재 확률 1 개를 의미하며, 이에 클래스 수 C를 더하여 각 셀에서 예측해야 하는 총 출력 차원을 결정한다.

2.2.2 YOLO의 내부 구성 요소

YOLO 의 핵심 구조는 객체의 위치 추정과 정확한 분류를 달성하기 위해 다양한 알고리즘적 요소들로 구성된다[6].

먼저, YOLO 는 기본적으로 합성곱 신경망(CNN)을 기반으로 동작하며, 입력 이미지로부터 시각적 특징(feature)을 추출한 뒤 이를 활용하여 객체의 위치와 종류를 예측한다. 최근의 YOLO 버전에서는 CSPDarknet, FPN(Feature Pyramid Network), PAN(Path Aggregation Network)과 같은 모듈이 백본(Backbone)으로 활용되어 특징 추출 성능을 더욱 향상시킨다.

다음으로, **Anchor Box** 는 다양한 크기와 비율의 객체를 탐지하기 위해 도입된 개념으로, 사전에 정의된 여러 개의 박스 형태를 기준으로 각 객체에 가장 적합한 박스를 선택하는 방식이다. 이를 통해 모델은 다양한 형태와 크기의 객체를 효과적으로 탐지할 수 있다.

IoU(Intersection over Union)는 예측된 바운딩 박스와 실제 객체와의 겹침 정도를 정량적으로 평가하는 지표로, 예측 위치의 정확도를 판단하는 데 사용된다. 일반적으로 설정된 임계값보다 낮은 IoU 값을 가진 예측은 오탐지로 간주된다.

또한, NMS(Non-Maximum Suppression)는 하나의 객체에 대해 중복된 바운딩 박스가 다수 예측되는 문제를 해결하기 위해 적용되는 기법이다. 예측된 박스들 중 신뢰도(confidence score)가 가장 높은 하나를 남기고, 나머지는 제거함으로써 중복 예측을 줄이고 결과의 명확성을 높인다.

마지막으로, 손실 함수(Loss Function)는 바운딩 박스의 위치 좌표, 객체 존재 여부, 클래스 분류 결과를 모두 고려한 형태로 구성된다. 위치 오차는 주로 평균 제곱 오차(Mean Squared Error, MSE)를 통해 계산되며, 클래스 분류는 교차 엔트로피 손실(Cross Entropy Loss)을 사용한다. 모델은 이러한 복합 손실 값을 최소화하는 방향으로 학습을 진행하여, 객체 탐지 성능을 점차 향상시켜 나간다.

2.2.3 YOLOv5의 특징

YOLOv5 는 YOLO 시리즈 중 비교적 최신 버전으로, PyTorch 프레임워크 기반으로 구현되어 있으며, 기존의 YOLOv4 에 비해 경량화와 속도 측면에서 향상된 성능을 제공한다. 모델은 여러 크기(nano, small, medium, large 등)로 제공되며, 환경에 따라 적절한 모델 선택이 가능하다.[6]

YOLOv5 는 CSP 구조를 이용한 백본(Backbone)과, FPN/PAN 을 조합한 Neck 구조를 사용한다. 이러한 계층적 설계는 다양한 크기의 객체를 동시에 탐지할 수 있게 해주며, 실시간 영상 분석에 효과적으로 사용된다.

YOLOv5 는 빠른 연산 속도와 우수한 정확도를 동시에 만족시키며, 객체 탐지가 요구되는 다양한 분야에서 널리 활용되고 있다.

2.2.4 전처리 및 후처리 기법

YOLO 모델은 학습 시 정해진 입력 형식과 데이터 분포를 기준으로 최적화되므로, 추론 시에도 학습과 유사한 입력 환경을 구성하는 것이 중요하다. 이를 위해 입력 이미지에 대해 특정한 전처리 과정을 수행하고, 모델의 예측 결과에 대해서는 후처리를 통해 실제 해석 가능한 형태로 정제한다.[7][8]

(1) Letterbox Resize

YOLO 모델은 고정된 크기의 입력 이미지를 요구하므로, 원본 이미지의 해상도를 유지하면서 크기를 맞추기 위해 Letterbox 방식의 resize 가 사용된다. 이는 비율을 유지한 채 축소한 뒤, 남는 영역에 padding 을 추가하는 방식이다.

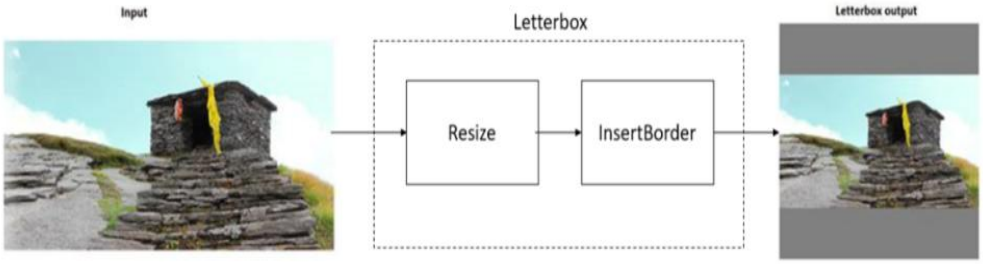
입력 이미지의 크기를 (w, h), 모델 입력 크기를 (W, H)라 할 때, 축소 비율 r은 다음과 같이 정의된다.

$$r = \min\left(\frac{W}{w}, \frac{H}{h}\right)$$

여백은 다음과 같이 계산된다.

$$p_x = \frac{W - r \cdot w}{2},\ p_y = \frac{H - r \cdot h}{2}$$

이미지는 이 비율로 축소된 뒤, p_x, p_y만큼 각각 좌우, 상하에 padding 이 적용되어 모델 입력에 맞춰진다.[9]



[그림 2] Letterbox 전처리 과정

(2) 바운딩 박스 형식 변환

YOLO 는 예측 결과를 중심 좌표 (x, y), 폭 w, 높이 h 형태로 출력하며, 이는 일반적인 사각형 좌표 형태인 (x_1, y_1, x_2, y_2)로 변환해야 한다. 변환 수식은 다음과 같다.

$$x_1 = x - \frac{w}{2}, \quad y_1 = y - \frac{h}{2}, \quad x_2 = x + \frac{w}{2}, \quad y_2 = y + \frac{h}{2}$$

(3) 좌표 스케일 복원

모델이 출력한 좌표는 전처리된 입력 이미지 기준이므로, 이를 원본 이미지 기준으로 되돌리는 보정 작업이 필요하다. 앞서 계산된 축소 비율 r 과 padding (p_x, p_y)을 이용하여, 다음과 같이 원래 좌표로 복원할 수 있다.

$$x_{\text{orig}} = r^{-1} \cdot (x_{\text{pred}} - p_x), \quad y_{\text{orig}} = r^{-1} \cdot (y_{\text{pred}} - p_y)$$

(4) 비최대 억제 (Non-Maximum Suppression)

YOLO 는 하나의 객체에 대해 중복된 예측을 할 수 있으므로, 이 중 신뢰도가

가장 높은 결과 하나만 남기고 나머지를 제거하는 비최대 억제(NMS) 과정을 수행한다. 이때 두 예측 박스 간의 중첩 정도는 다음의 IoU(Intersection over Union)으로 계산된다.

$$\mathrm{IoU}(A, B) = \frac{\mathrm{Area}(A \cap B)}{\mathrm{Area}(A \cup B)}$$

IoU 가 설정된 임계값(보통 0.5~0.6)을 초과하면, 해당 박스들은 동일 객체로 간주되며 가장 높은 confidence score 를 가진 박스만 유지된다.

이와 같은 전처리 및 후처리 단계는 YOLO 모델이 학습 당시와 동일한 입력 형식을 유지하고, 출력 결과를 실질적으로 활용 가능한 형태로 정제하기 위해 필수적으로 수행된다.

2.3 Kafka 메시징 시스템

Apache Kafka 는 대용량 실시간 데이터 스트리밍을 처리하기 위한 오픈소스 분산 메시징 플랫폼이다. 본래는 LinkedIn 에서 로그 수집 시스템을 목적으로 개발되었으며, 현재는 다양한 산업 분야에서 데이터 파이프라인 구축, 로그 수집, 이벤트 처리, 실시간 분석 등의 용도로 폭넓게 활용되고 있다. Kafka 는 높은 처리량과 내결함성(fault-tolerance), 확장성(scalability)을 동시에 제공하는 구조를 갖추고 있어, 실시간 분산 시스템의 핵심 구성 요소로 자리 잡고 있다.

Kafka 의 기본 구성 요소에는 Producer, Broker, Consumer, Topic, Partition, 그리고 Zookeeper 가 있다[10]

Producer 는 Kafka 로 메시지를 전송하는 역할을 하며, 이는 일반적으로 센서, 애플리케이션, 데이터 수집 지점 등과 같은 데이터 소스가 담당한다.

Broker 는 Kafka 의 중앙 서버로서, 수신된 메시지를 저장하고 이를 Consumer 에게 전달하는 기능을 수행한다. 하나의 Kafka 클러스터는 다수의 Broker 로 구성될 수 있으며, 이를 통해 고가용성과 부하 분산이 가능하다.

Consumer 는 Kafka 에서 메시지를 구독하고 처리하는 역할을 하며, 사용자는

특정 Topic 을 기준으로 필요한 데이터를 지속적으로 수신할 수 있다.

Topic 은 메시지가 게시(publish)되고 구독(subscribe)되는 논리적 단위로, Kafka 내의 모든 데이터는 특정 Topic 에 속한다. Producer 는 특정 Topic 에 메시지를 전송하고, Consumer 는 해당 Topic 을 구독하여 메시지를 수신하게 된다. 각 Topic 은 하나 이상의 Partition 으로 분할될 수 있으며, 각 Partition 은 순차적이고 독립적인 로그 구조를 가진다. 이를 통해 메시지의 병렬 처리와 시스템의 수평 확장이 가능해진다.

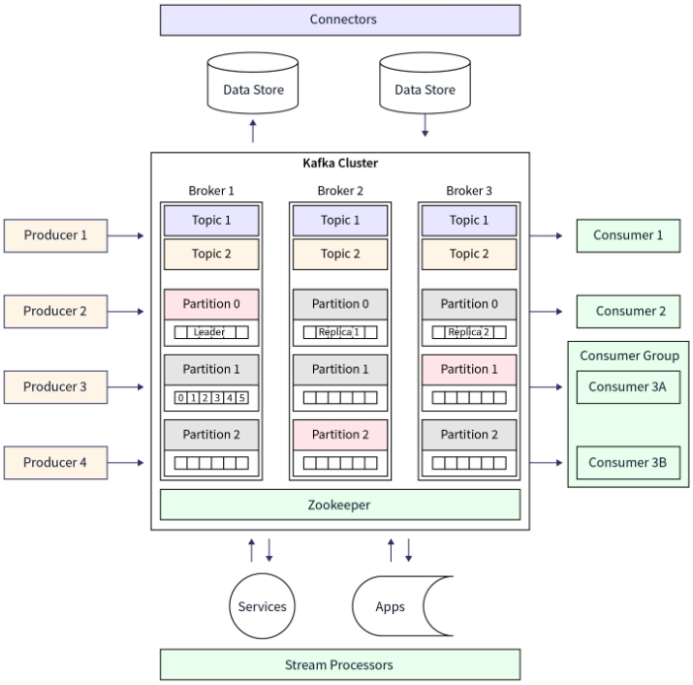
Kafka 2.x 버전 이하에서는 Zookeeper 를 통해 클러스터 상태를 관리하고 Broker 간 메타데이터를 유지했으나, Kafka 3.0 이상에서는 이를 대체하기 위해 KRaft(Kafka Raft) 모드가 도입되어 별도의 외부 의존성 없이 자체 클러스터 메타데이터 관리가 가능해졌다.

Kafka 는 메시지를 디스크에 저장하면서도 배치 처리 없이 실시간 전송이 가능하므로, 데이터 신뢰성과 전송 속도를 동시에 만족시킨다. 메시지는 일정 기간 동안 저장되기 때문에, Consumer 가 일시적으로 오프라인 상태에 있더라도 이후에 복구하여 메시지를 재수신할 수 있다. 기본적으로 Kafka 는 at-least-once 메시지 전달을 보장하며, 설정에 따라 exactly-once 처리도 가능하여 높은 신뢰성이 요구되는 환경에도 적용할 수 있다.

Kafka 의 핵심 처리 흐름은 다음과 같다:

1. Producer 가 특정 Topic 에 메시지를 전송한다.
2. Kafka Broker 는 메시지를 해당 Topic 의 Partition 에 저장한다.
3. Consumer 는 자신이 구독한 Topic 에서 메시지를 순차적으로 수신하고 처리한다.

이 구조는 단순하지만 매우 강력하며, 데이터 생산과 소비를 완전히 분리하여 다양한 처리 시스템 간의 유연한 연동을 가능하게 한다.[11]



[그림 3] Kafka 구조

2.4 RTSP 스트리밍과 OpenCV 기반 영상 처리

YOLO 기반 객체 탐지 모델을 실시간 영상에 적용하기 위해서는, 외부 영상 소스로부터 연속적인 이미지 데이터를 안정적으로 수신하고 처리하는 입력 시스템이 필요하다. 이러한 실시간 영상 입력은 주로 RTSP(Real Time Streaming Protocol)를 통해 이루어지며, 영상 처리 도구로는 OpenCV 가 널리 활용된다.

RTSP 는 실시간 멀티미디어 스트리밍을 위한 표준 프로토콜로, 주로 CCTV, IP 카메라, NVR(Network Video Recorder) 등에서 사용된다. RTSP 는 HTTP 와는 달리 지속적인 연결을 유지하며, 비디오와 오디오 데이터를 실시간으로 전송할 수 있도록 설계되어 있다. RTSP 는 자체적으로 데이터를

전송하지 않고, 데이터 전송은 RTP(Real-time Transport Protocol)나 RTCP 를 통해 수행되며, RTSP 는 이들의 세션 제어 역할을 담당한다.[12]

영상 스트리밍 주소는 일반적으로 다음과 같은 형식을 따른다.

rtsp://[사용자명]:[비밀번호]@[IP주소]:[포트]/[경로]

예를 들어, rtsp://admin:1234@192.168.0.10:554/stream1 와 같은 주소를 통해 특정 IP 카메라의 실시간 영상을 수신할 수 있다.

RTSP 스트리밍으로부터 영상 프레임을 획득하기 위해 OpenCV 의 cv2.VideoCapture() 함수를 사용한다. 이 함수는 RTSP 주소를 입력받아 연결을 생성하고, read() 메서드를 통해 실시간으로 프레임을 추출할 수 있다.

프레임 추출 과정은 다음과 같이 진행된다:

1. RTSP 주소로부터 비디오 스트림 연결 생성
2. 일정 시간 간격으로 read() 함수를 호출하여 프레임 수신
3. 수신된 프레임을 정지 이미지 형태로 저장하거나 후속 처리 모델에 전달

OpenCV 는 수신된 프레임을 NumPy 배열 형태로 반환하며, 이는 딥러닝 모델에 바로 입력할 수 있도록 전처리(크기 조정, 정규화 등)를 적용할 수 있게 한다. 또한 OpenCV 는 영상 수신 오류, 끊김, 딜레이 발생 시 예외 처리를 위한 기능도 함께 제공한다.

2.5 FastAPI 기반 결과 수신 시스템

FastAPI 는 Python 기반의 고성능 웹 프레임워크로, RESTful API 서버를 빠르고 간결하게 구축할 수 있도록 설계되었다. Starlette 과 Pydantic 을 기반으로 하며, 비동기(Asynchronous) 요청 처리를 기본적으로 지원하여 높은 응답성과 처리 속도를 제공한다.[13]

]

FastAPI 의 가장 큰 특징은 자동 문서화(OpenAPI/Swagger UI), 타입 기반 유효성 검사, 최소한의 코드로 구성 가능한 라우팅 시스템이다. 특히 JSON 형식의 데이터 요청 및 응답을 매우 효율적으로 처리할 수 있어, 실시간 객체 탐지 결과를 외부 모듈로부터 안정적으로 수신하는 데 적합하다.

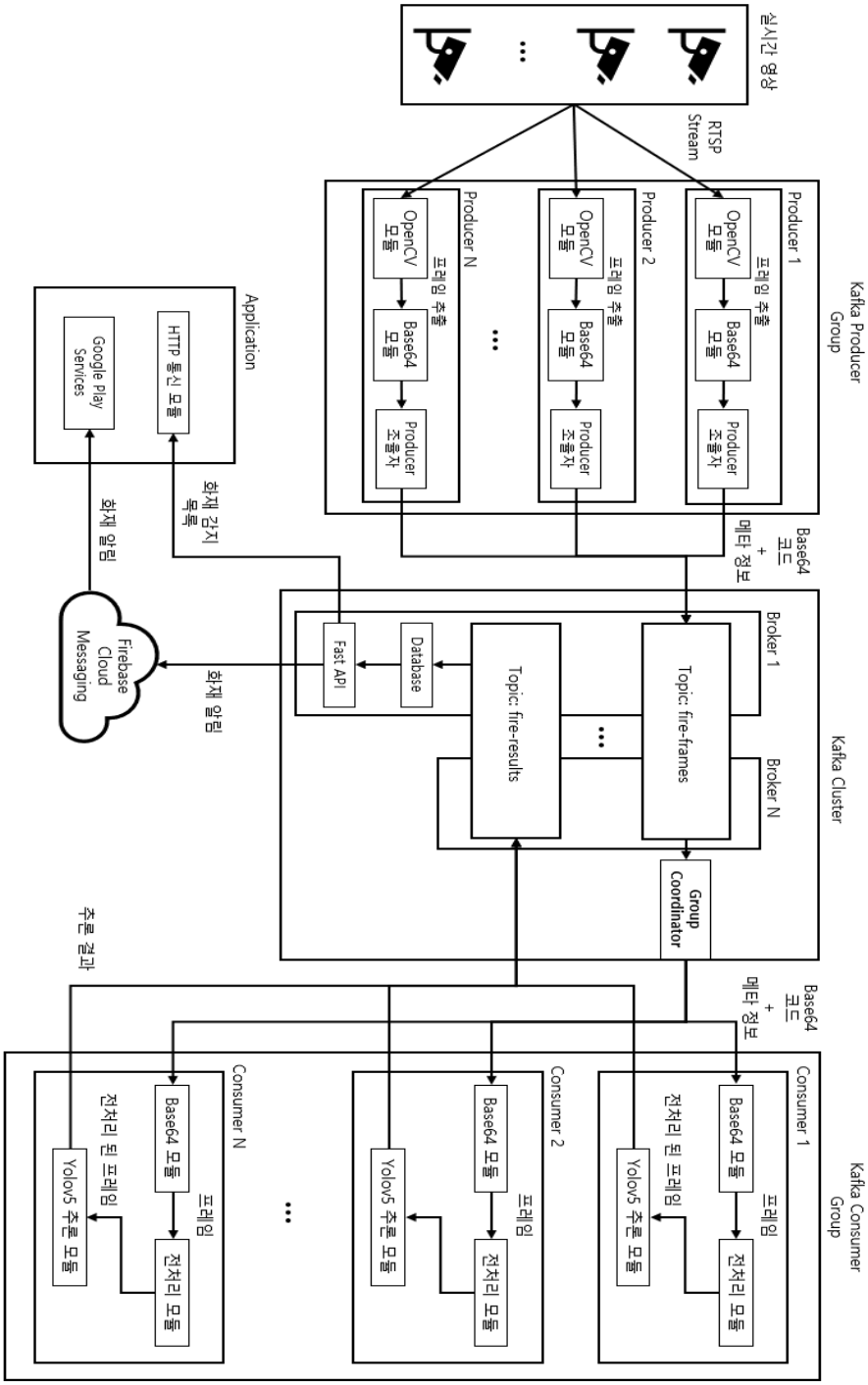
이 프레임워크는 서버 부하가 적고 유지 보수가 쉬워, 감지 결과 수신과 같은 이벤트 중심의 통신 환경에 효과적으로 적용될 수 있다. 또한 Python 기반의 다른 비동기 기술 및 데이터베이스, 메시징 시스템과의 연동이 수월하다는 점에서, 소규모 또는 중간 규모의 실시간 처리 시스템에서 많이 사용된다.

제 3 장 시스템 설계 및 구현

본 장에서는 Kafka 기반 분산 아키텍처를 활용한 영상 객체 탐지 기반 화재 감지 시스템의 설계 및 구현 과정을 설명한다. 3.1 절에서는 YOLOv5 기반 객체 탐지 모델의 설계 및 구현, 3.2 절에서는 Kafka 를 중심으로 한 메시지 처리 흐름과 탐지 결과 저장 구조를 설명한다. 3.3 절에서는 감지 이력을 사용자에게 제공하고 알림을 전달하기 위한 FastAPI 서버와 Android 애플리케이션의 설계를 기술한다.

시스템은 RTSP 기반 CCTV 스트림을 OpenCV 로 수신하여 Kafka Producer 가 프레임 단위로 처리하고, 이를 fire-frames 토픽을 통해 Kafka Broker 로 전송하는 것으로 시작된다. 이후 Kafka Consumer 는 해당 토픽을 구독하여 YOLOv5 모델로 프레임을 실시간 분석하며, 화재가 감지된 경우 결과를 fire-results 토픽으로 재전송한다. 이 감지 결과는 별도의 저장 모듈에 의해 SQLite 데이터베이스에 기록되며, FastAPI 는 Kafka 브로커 중 하나가 실행 중인 컴퓨터에 배포되어 Android 애플리케이션과의 통신 기능을 담당한다. 사용자는 FastAPI 서버를 통해 감지 이력을 조회하거나, Firebase Cloud Messaging 을 통해 실시간 알림을 수신할 수 있다.

시스템 전체 흐름에 대한 구조는 [그림 4]와 같다.


실시간 영상
RTSP Stream
Kafka Producer Group
Producer 1
OpenCV 모듈
프레임 추출
Base64 모듈
Producer 조율자
Producer 2
OpenCV 모듈
프레임 추출
Base64 모듈
Producer 조율자
Producer N
OpenCV 모듈
프레임 추출
Base64 모듈
Producer 조율자
Base64 코드 + 메타 정보
Kafka Cluster
Broker 1
Topic: fire-frames
Broker N
Topic: fire-results
Database
Fast API
Group Coordinator
Application
HTTP 통신 모듈
Google Play Services
화재 감지 등록
화재 알림
Firebase Cloud Messaging
화재 알림
추론 결과
Base64 코드 + 메타 정보
Kafka Consumer Group
Consumer 1
Base64 모듈
프레임
전처리 모듈
전처리 된 프레임
Yolov5 추론 모듈
Consumer 2
Base64 모듈
프레임
전처리 모듈
Yolov5 추론 모듈
Consumer N
Base64 모듈
프레임
전처리 모듈
전처리 된 프레임
Yolov5 추론 모듈

[그림 4] 시스템 설계

3.1 YOLOv5 기반 객체 탐지 모델의 설계 및 구현

본 논문에서는 실시간 영상 기반 화재 감지를 위해 YOLOv5 객체 탐지 모델을 사용하였다. CCTV 에서 수집한 영상 데이터를 기반으로 전처리 과정을 수행한 뒤, 화재 관련 객체(불꽃, 연기)를 인식할 수 있도록 딥러닝 모델을 학습하였다. 아래 항에서는 학습 데이터의 구성, 전처리 과정, 모델 학습 설정, 그리고 학습된 모델의 적용 방식에 대해 설명한다.

3.1.1 학습 데이터 수집

YOLOv5 모델의 학습을 위해 사용된 데이터는 과학기술정보통신부와 한국지능정보사회진흥원(NIA)이 운영하는 AI Hub 에서 제공하는 "화재 발생 예측 영상 (고도화)- 영상 기반 화재 감시 및 발생 위치 탐지 데이터"를 사용하였다. 해당 데이터셋은 실제 화재 현장을 촬영한 영상 클립으로부터 추출된 이미지로 구성되어 있으며, 불꽃 및 연기와 같은 화재 현상이 포함된 장면을 다수 포함하고 있다.

전체 데이터는 약 5,304 개의 12 초 분량 클립으로부터 생성된 약 190 만 장의 이미지로 구성되어 있으며, 다양한 장소, 시간대, 촬영 각도에서 수집된 영상이 포함되어 있다. 촬영 환경은 실내·외 구역, 야간·주간 조건, 다양한 구조물과 배경을 포함하고 있으며, 각 이미지에는 객체의 위치 정보를 바운딩 박스 형태로 라벨링한 COCO 형식의 JSON 파일이 함께 제공된다.[14]

데이터는 세 가지 클래스로 분류되며, 화재(불꽃) 이미지가 전체의 약 40%, 연기 이미지가 약 40%, 정상(비화재) 상태의 이미지가 약 20%를 차지한다.

3.1.2 학습 데이터 전처리 및 가공

AI Hub 에서 제공하는 화재 객체 인식 데이터셋은 영상 클립을 프레임 단위로 분할하여 생성된 이미지들로 구성되어 있으며, 이로 인해 유사한 장면이

연속적으로 포함된 이미지가 다수 존재한다. 이러한 중복 프레임은 모델 학습 시 불필요한 반복 학습을 유발할 수 있기 때문에, 본 연구에서는 프레임 간 유사성을 줄이기 위한 데이터 축소 과정을 적용하였다.

각 영상 클립으로부터 생성된 이미지 중 4 프레임마다 1 장의 이미지를 추출하는 방식으로 데이터를 구성하였으며, 이를 통해 전체 데이터 수를 줄이면서도 주요 장면 정보는 유지되도록 하였다. 이렇게 구성된 이미지들은 학습, 검증, 테스트 세트로 분할되어 모델 학습에 사용되었다. 최종적으로 구성된 데이터셋의 클래스 분포 및 분할 비율은 다음 [표 1]과 같다.

[표 1] 데이터 구분

데이터 유형	클래스	이미지 수	비율 (%)
학습용	화재	152,712	40%
학습용	연기	152,712	40%
학습용	정상	76,356	20%
소계	-	381,780	100%
검증용	화재	19,080	40%
검증용	연기	19,080	40%
검증용	정상	9,540	20%
소계	-	47,700	100%
총합	-	429,480	-

또한, 모델 학습에 앞서 모든 입력 이미지는 YOLOv5 구조에 맞게 고정 입력 크기(640×640)에 맞춰 조정되었다. 이 과정에서는 이미지의 원래 비율을 유지하고 부족한 영역에 패딩을 추가하는 letterbox 방식의 전처리가 적용되었으며, 이는 앞서 2.2.2 절에서 설명한 방식과 동일하다. 해당 전처리는

입력 영상의 왜곡을 방지하면서 다양한 해상도의 이미지에 대해 일관된 입력 형태를 유지할 수 있도록 한다.

3.1.3 모델 학습 환경 및 설정

YOLOv5 모델은 PyTorch 기반으로 구현되었으며, 본 연구에서는 Ultralytics 에서 제공하는 YOLOv5s 사전 학습 가중치를 기반으로 화재 객체 탐지 모델을 학습하였다. 모델 학습은 Windows 운영체제를 사용하는 로컬 컴퓨터에서 수행되었으며, 해당 시스템은 NVIDIA GeForce RTX 4060 GPU(8GB VRAM), Intel Core i7 프로세서, 32GB RAM 환경으로 구성되었다. 소프트웨어 환경은 Python 3.9, PyTorch 2.0, CUDA 11.8 을 기반으로 하였다.

학습에 사용된 주요 하이퍼파라미터는 다음과 같다. 입력 이미지 크기는 640×640 으로 고정되었으며, 배치 크기는 16, 학습 에폭 수는 10 으로 설정하였고 학습률의 경우에는 YOLOv5 의 기본 설정 값인 0.01 에서부터 최종 학습률 0.001 로 설정하였다. 해당 설정은 하드웨어의 처리 성능과 데이터셋의 규모를 고려하여 선택되었다. 학습에는 클래스 정보와 데이터 경로를 포함한 YAML 형식의 설정 파일이 사용되었으며, YOLOv5 에서 제공하는 학습 스크립트를 통해 실행되었다.

학습 도중 각 에폭마다 검증 데이터셋을 기준으로 성능 평가가 수행되었고, 학습 완료 후 생성된 모델 가중치는 추론용으로 저장되어 분산 처리 시스템의 Kafka Consumer 모듈에 통합되었다. 이를 통해 실시간으로 수신되는 영상 프레임에 대해 자동으로 객체 탐지가 수행될 수 있도록 구성되었다.

3.1.4 모델 저장 및 추론 활용 방식

YOLOv5 모델의 학습이 완료되면 최종 가중치는 PyTorch 에서 사용하는 .pt 형식으로 저장된다. 본 연구에서는 학습이 완료된 모델 가중치를 후속 시스템에 연동하여 실시간 객체 탐지를 수행할 수 있도록 구성하였다. 저장된 모델은

PyTorch 의 torch.load() 함수를 통해 불러와지고, 추론 시에는 eval() 모드로 전환되어 사용된다.

해당 모델은 Kafka 기반 분산 처리 구조의 Consumer 모듈에 탑재되었으며, 이 모듈은 수신한 영상 프레임에 대해 실시간으로 객체 탐지를 수행하는 역할을 한다. 모델은 학습과 동일한 입력 크기(640×640)를 유지하도록 설정되었고, 추론 결과의 신뢰도 기준은 confidence threshold 0.25 로 고정되었다.
해당 threshold 값의 선택 배경과 성능에 미치는 영향은 후속 장에서 실험 결과와 함께 분석한다.

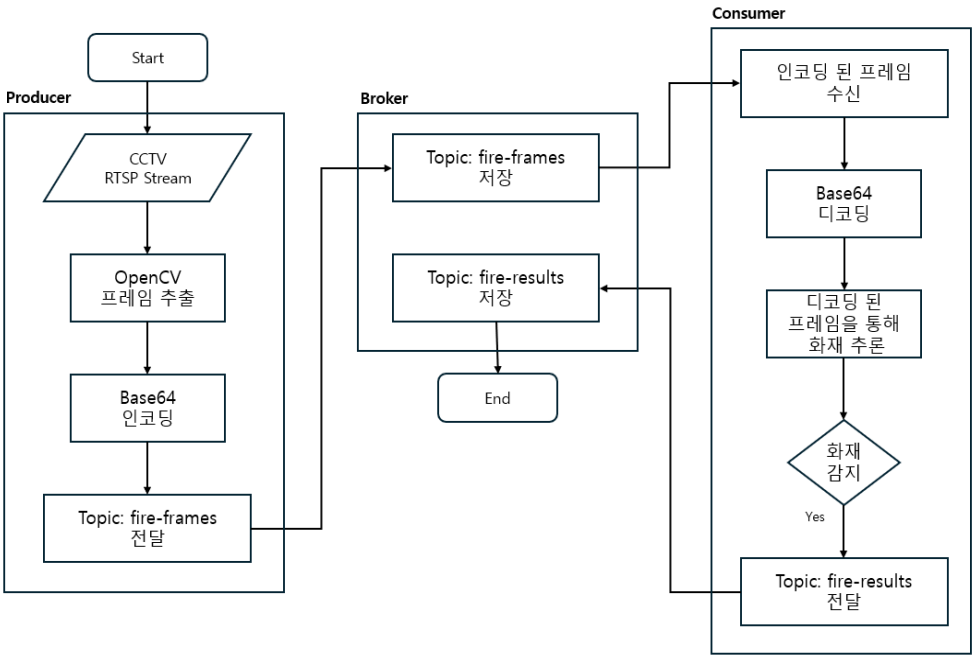
모델이 적용된 Consumer 는 시스템의 일부로 통합되어 있으며, 실시간 영상 데이터를 처리하여 감지 결과를 산출하는 데 활용된다. 추론 처리 방식의 구체적인 흐름과 Kafka 와의 연계 구조는 3.2 절에서 자세히 설명한다.

3.2 Kafka 기반 분산 시스템 설계 및 데이터 흐름

본 시스템은 실시간 CCTV 영상을 기반으로 객체 탐지를 수행하기 위해, 데이터 전송과 처리를 효과적으로 분산시킬 수 있는 구조로 Apache Kafka 를 활용하였다. Kafka 는 대규모 실시간 데이터 스트리밍에 최적화된 메시지 브로커 시스템으로, Producer, Broker, Consumer 간의 느슨한 결합 구조를 통해 높은 확장성과 처리 안정성을 제공한다.

본 절에서는 Kafka 가 본 시스템 내에서 수행하는 역할과 전체 구성 요소를 기술하며, 각 메시지의 흐름을 따라 데이터가 어떻게 전달되고 처리되는지를 설명한다. 이를 위해 3.2.1 절에서는 Kafka 의 기본 구성 요소와 시스템 내 역할을 설명하고, 3.2.2 절에서는 본 시스템에서 사용된 토픽 구조에 대해 다룬다. 3.2.3 절에서는 메시지 흐름을 구체적으로 설명한다.

Kafka 구성 요소 간의 실제 데이터 흐름은 [그림 5]에 나타낸다.



[그림 5] Kafka 기반 화재 감지 시스템의 메시지 처리 흐름

3.2.1 Kafka 구성 요소 및 역할

Apache Kafka 는 분산 로그 스트리밍 플랫폼으로, 데이터 생산자(Producer)와 소비자(Consumer) 간의 메시지 교환을 중개하는 역할을 수행한다. Kafka 의 핵심 구성 요소는 Producer, Broker, Topic, Consumer 로 나뉘며, 이들 각각은 본 시스템에서 다음과 같은 역할을 수행한다.

Producer 는 외부에서 생성된 데이터를 Kafka 에 전달하는 역할을 하며, 본 시스템에서는 RTSP 스트림을 OpenCV 를 통해 프레임 단위로 분할하고, 이를 Kafka 에 전송하는 모듈이 Producer 로 동작한다. Producer 는 영상 프레임을 base64 로 인코딩한 후, 이를 fire-frames 라는 토픽에 메시지 형태로 전송한다.

Broker 는 Kafka 클러스터 내에서 데이터를 유지하고, 토픽을 관리하며 Producer 와 Consumer 간 메시지를 중개한다. 본 시스템에서는 Broker 가 수신된 영상 프레임과 탐지 결과 메시지를 내부 토픽별로 관리한다.

Topic 은 Kafka 내에서 메시지를 분류하는 논리적 채널이며, 각 메시지는 특정 토픽에 게시된다. 시스템에는 프레임 전달용 토픽인 fire-frames 와, 탐지 결과 전달용 토픽인 fire-results 가 존재하며, 데이터의 흐름과 목적에 따라 구분되어 사용된다.

Consumer 는 특정 토픽을 구독하여 메시지를 수신하고 처리하는 역할을 하며, 본 시스템에서는 YOLOv5 모델이 탑재된 Consumer 가 fire-frames 토픽을 구독하여 수신된 프레임을 분석한다. 화재가 감지되면 감지 결과를 정리하여 fire-results 토픽으로 다시 전송하며, 이 결과는 후속 처리 시스템에서 활용된다.

Kafka 는 이러한 구성 요소들이 비동기적으로 통신할 수 있도록 하여, 시스템 전체가 느슨하게 연결되면서도 각 모듈이 독립적으로 동작할 수 있도록 한다. 이를 통해 처리량이 높은 실시간 영상 데이터를 안정적으로 수신, 처리, 분산할 수 있는 기반을 제공한다.

3.2.2 토픽 구조 설계

Kafka 를 기반으로 한 본 시스템에서는 메시지의 종류와 처리 단계를 명확히 구분하기 위해 두 개의 토픽을 구성하였다. 각각은 시스템 내에서 프레임 전달과 탐지 결과 전달이라는 서로 다른 역할을 수행하며, 처리 모듈 간 데이터 흐름을 분리하고 모듈화된 구조를 유지하는 데 기여한다.

첫 번째 토픽은 fire-frames 로, CCTV 영상 스트림으로부터 추출된 프레임이 전송되는 채널이다. Kafka Producer 는 RTSP 스트림에서 OpenCV 를 이용해 일정 간격으로 프레임을 추출하고, 이를 JPEG 형식의 이미지로 변환한 뒤 base64 로 인코딩하여 메시지로 생성한다. 해당 메시지에는 이미지 외에도 프레임의 시각적 정보나 카메라 ID 등 간단한 메타데이터가 함께 포함된다. 이 데이터는 Kafka 를 통해 전송된 후, Consumer 가 구독하여 객체 탐지를 위한 입력으로 사용한다.

두 번째 토픽은 fire-results 이며, YOLOv5 모델이 수행한 객체 탐지 결과를 전송하는 데 사용된다. Consumer 는 fire-frames 로부터 수신한 프레임을 분석한

뒤, 화재가 감지된 경우에만 탐지 결과를 JSON 형식으로 구조화하여 fire-results 토픽에 게시한다. 해당 메시지에는 바운딩 박스 처리된 사진과 위치, 타임스탬프에 대한 정보가 포함된다.

이러한 토픽 분리는 데이터 흐름을 명확히 구분하고, 특정 Consumer 가 처리해야 하는 메시지 종류를 명확히 지정할 수 있게 해준다. 또한, 이후 단계에서 결과 저장, 사용자 알림, 대시보드 시각화 등 별도의 후속 처리를 담당하는 모듈이 fire-results 토픽만을 구독함으로써 시스템 확장이 용이해진다.

3.2.3 메시지 처리 흐름

본 시스템에서 화재 감지를 위한 전체 데이터 흐름은 Kafka 를 중심으로 구성된 비동기 메시지 처리 구조를 기반으로 한다. 처리 흐름은 실시간 CCTV 영상이 Kafka Producer 에 입력되는 시점부터, Kafka Broker 를 통해 전달되고, 최종적으로 Kafka Consumer 가 객체 탐지를 수행한 후 탐지 결과를 다시 Kafka 에 게시하는 단계로 구성된다.

우선, Kafka Producer 는 RTSP 프로토콜을 통해 CCTV 로부터 실시간 영상 스트림을 수신한다. 이 스트림은 OpenCV 를 이용해 프레임 단위로 분할되며, 각 프레임은 JPEG 포맷 이미지로 저장된 뒤 base64 로 인코딩된다. 인코딩된 이미지 데이터는 간단한 메타데이터(예: 카메라 ID, 타임스탬프 등)와 함께 fire-frames 토픽에 메시지로 게시된다.

Kafka Broker 는 해당 메시지를 수신한 뒤 내부적으로 관리하며, fire-frames 토픽을 구독하고 있는 Consumer 에게 메시지를 전달한다. 각 Kafka Consumer 는 base64 로 인코딩된 프레임을 디코딩한 후, YOLOv5 모델을 통해 객체 탐지를 수행한다. 탐지 결과는 바운딩 박스 처리된 사진과 위치, 타임스탬프 정보로 구성되며, JSON 형식으로 구조화된다.

이 결과는 fire-results 토픽으로 다시 Kafka 에 게시되며, 해당 토픽은 후속 처리 모듈(FastAPI 서버, 알림 시스템 등)에서 구독하여 데이터베이스 저장, 사용자 알림, 통계 기록 등의 작업에 활용된다.

이와 같은 처리 흐름은 Kafka 의 비동기 메시징 구조에 기반하고 있어 각 단계가 독립적으로 동작할 수 있으며, 시스템 부하가 분산되고 확장성이 확보되는 구조를 형성한다.

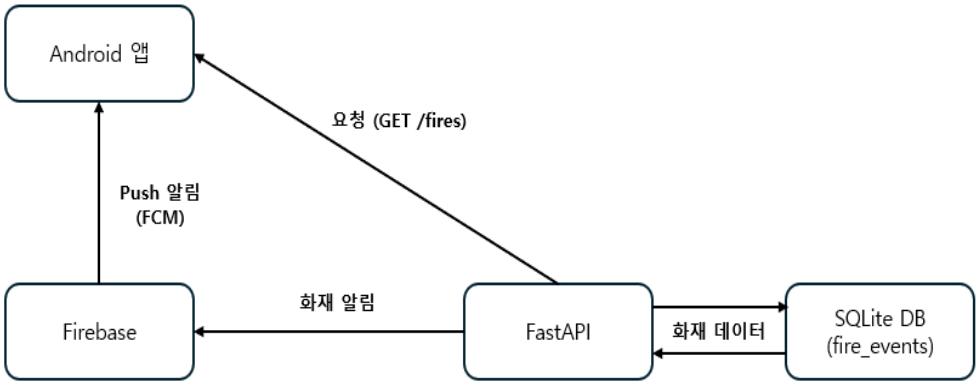
3.3 FastAPI 서버 및 Android 애플리케이션 설계

본 시스템은 실시간 화재 감지 결과를 사용자에게 전달하고, 감지 이력을 확인할 수 있도록 사용자 인터페이스를 제공한다. 이를 위해 Python 기반의 웹 프레임워크인 FastAPI 를 이용한 경량 서버와 Android 모바일 애플리케이션을 함께 설계하였다. 두 구성 요소는 YOLOv5 모델이 분석한 결과를 최종적으로 사용자에게 전달하는 역할을 수행한다.

FastAPI 는 감지 결과가 저장된 데이터베이스에 접근하여 RESTful API 를 통해 Android 애플리케이션의 요청에 응답한다. 서버는 Kafka 와 직접 연결되지 않고, Kafka Consumer 가 감지 결과를 저장한 로컬 데이터베이스(SQLite)와만 연동된다. 이 구조는 실시간성과 시스템 분리를 모두 고려한 방식으로, 분석 결과를 안전하게 저장하고 선택적으로 사용자에게 제공할 수 있도록 한다.

Android 애플리케이션은 FastAPI 서버로부터 전달받은 데이터를 기반으로 감지 결과를 리스트 형태로 표시하며, 각 결과에는 탐지 시간, 위치, 객체 정보 및 이미지 썸네일이 포함된다. 또한 Firebase Cloud Messaging(FCM)을 통해 실시간 푸시 알림을 수신하여 사용자에게 즉각적인 경고 기능을 제공한다.

이러한 구조는 분석 결과를 서버를 통해 안정적으로 저장하고, 사용자에게 실시간으로 정보를 전달하는 전체 흐름을 구성한다.
FastAPI 서버와 Android 앱 간의 데이터 전달 방식은 [그림 6]에 나타낸다.



[그림 6] FastAPI 서버와 Android 애플리케이션 간의 데이터 흐름

이 절에서는 FastAPI 서버의 구성 및 역할을 3.3.1 절에서 설명하고, Android 애플리케이션의 구조와 기능은 3.3.2 절에서 상세히 기술한다.

3.3.1 FastAPI 서버 구성 및 역할

FastAPI 서버는 Kafka Consumer 가 저장한 화재 감지 결과를 외부 애플리케이션에서 조회할 수 있도록 RESTful API 를 제공하는 역할을 수행한다. 서버는 경량 구조를 가지며, 감지 결과가 저장된 SQLite 데이터베이스에 비동기 방식으로 접근하여 데이터를 응답 형식에 맞게 가공한 후 반환한다.

서버는 /fires 경로로 GET 요청을 수신하며, fire_events 테이블에 저장된 감지 결과를 모두 조회하여 클라이언트에게 반환한다. 데이터는 timestamp, camera_id, latitude, longitude, annotated_image 의 다섯 가지 필드로 구성되어 있으며, 결과는 감지 시각을 기준으로 내림차순 정렬되어 반환된다.

특히 감지된 화재 이미지(annotated_image)는 원본 바이너리 형태로 저장되어 있으며, API 응답 시에는 이를 base64 형식으로 인코딩하여 문자열 형태로 변환한 후 JSON 필드로 포함된다. 이를 통해 Android 애플리케이션과 같은 클라이언트가 이미지 데이터를 별도의 처리 없이 바로 활용할 수 있도록 하였다.

데이터베이스 연결은 aiosqlite 라이브러리를 활용하여 비동기 방식으로 구현되었으며, 이는 경량 웹 서버 구조에서도 높은 응답성과 안정적인 데이터 처리를 가능하게 한다. 전체 서버는 FastAPI 애플리케이션 인스턴스 하나로 구성되어 있으며, 별도의 Kafka 연결 없이 데이터베이스에 저장된 결과만을 대상으로 작동한다.

3.3.2 Android 애플리케이션 구성 및 기능

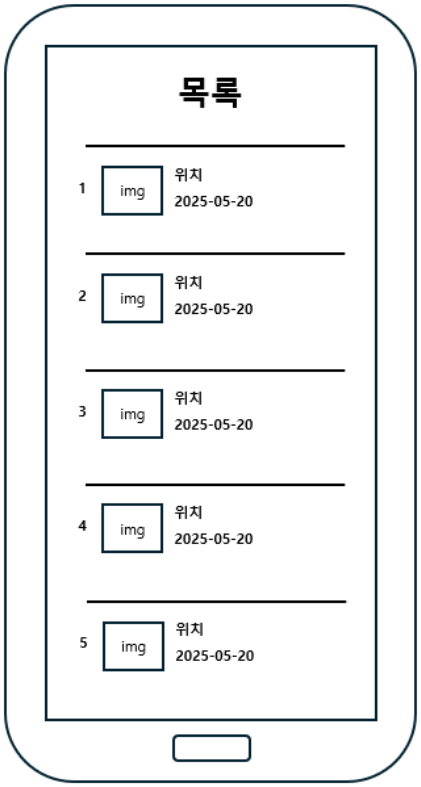
Android 애플리케이션은 FastAPI 서버로부터 화재 감지 이력을 조회하고, 사용자가 이를 시각적으로 확인할 수 있도록 인터페이스를 제공한다. 또한 Firebase Cloud Messaging(FCM)을 통해 실시간 화재 감지 알림을 수신함으로써 사용자에게 즉각적인 경고 기능을 제공한다.

사용자는 메인 화면에서 "감지 이력 조회" 버튼을 누르면 ListActivity 가 실행되며, 이 화면에서는 FastAPI 서버의 /fires 엔드포인트에 HTTP GET 요청을 보내 감지 결과를 수신한다. 서버로부터 전달받은 응답은 JSON 배열 형태로 구성되며, 각 항목에는 감지 시각, 카메라 ID, 위치 정보(위도·경도), 그리고 Base64 인코딩된 화재 이미지가 포함된다. 이 데이터는 FireDetection 클래스에 매핑되어 ArrayList 형태로 저장된다.

이후 FireDetectionAdapter 를 통해 각 감지 결과는 RecyclerView 에 리스트 형태로 렌더링되며, 각 항목에는 감지 일시, 위치 정보, 식별 번호, 이미지 썸네일이 함께 표시된다. 이미지 데이터는 Base64 에서 Bitmap 으로 디코딩되어 이미지뷰에 출력된다.

알림 기능은 FirebaseMessagingService 를 상속한 MyFirebaseMessaging-Service 클래스에서 구현되며, Firebase 서버로부터 전달되는 푸시 메시지를 수신해 알림 채널을 통해 사용자에게 즉각적으로 시각적 경고를 표시한다. Android 13 이상에서는 알림 권한 요청도 함께 처리된다.

Android 애플리케이션은 감지 결과 데이터를 리스트 형태로 사용자에게 제공한다. 각 항목은 이미지 썸네일, 위치 정보, 감지 일시로 구성되며, 사용자는 이를 스크롤 형태로 확인할 수 있다. 전체 감지 이력은 RecyclerView 를 통해 구성되어 있으며, 이를 통해 효율적인 목록 처리와 데이터 시각화를 지원한다.
감지 결과 리스트의 UI 구성은 [그림 7]과 같다.



[그림 7] Android 애플리케이션 내 감지 결과 목록 UI 구성

이와 같이 Android 애플리케이션은 FastAPI 와의 연동을 통해 감지 데이터를 실시간으로 수신 및 시각화하며, 알림 시스템을 통해 사용자에게 화재 정보를 능동적으로 전달할 수 있는 구조로 설계되었다.

제 4 장 실행 및 결과 분석

4.1 성능 평가 기준

객체 탐지 기반 화재 감지 시스템의 성능을 객관적으로 평가하기 위해, 본 연구에서는 Precision(정밀도), Recall(재현율), 그리고 mean Average Precision(mAP)를 기준으로 모델의 성능을 측정하였다. 해당 평가지표들은 객체 탐지 결과의 신뢰도와 탐지율을 수치화하여 모델의 실효성을 정량적으로 판단하는 데 사용된다.[15]

정밀도(Precision)는 모델이 화재로 예측한 결과 중 실제 화재였던 비율을 의미하며, 재현율(Recall)은 실제 화재 중에서 모델이 올바르게 탐지한 비율을 나타낸다. 두 지표는 다음과 같이 정의된다:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}$$

YOLOv5 는 이러한 탐지 정확도를 기반으로 평균 정밀도(AP)를 클래스별로 산출하며, 전체 클래스의 평균값을 mAP(mean Average Precision)로 제공한다. AP 는 Precision–Recall 곡선의 면적으로 정의되며, mAP 는 다음과 같이 계산된다:

$$\text{mAP} = \frac{1}{N} \sum_{i=1}^{N} \text{AP}_i$$

본 모델은 smoke(sm), fire(fl) 두 개의 탐지 클래스를 기준으로 학습되었으며, 검증 데이터 47,700 장에 대해 다음과 같은 성능을 보였다.

4.2 객체 탐지 정확도 평가

YOLOv5 기반으로 학습된 객체 탐지 모델은 화재(fire) 및 연기(smoke) 클래스를 대상으로 학습되었으며, 총 47,700 장의 검증 데이터에 대해 탐지 성능을 평가하였다. 본 절에서는 Precision, Recall, mAP 등의 지표를 기반으로 모델의 성능을 정량적으로 분석하고, confidence threshold 에 따른 변화 양상을 다양한 곡선을 통해 시각적으로 제시한다.

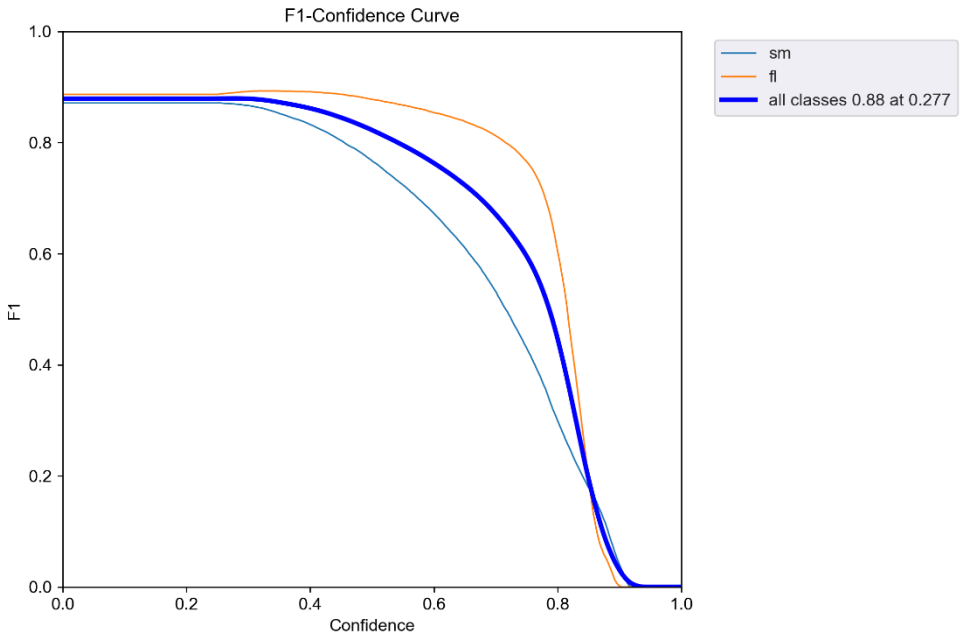
모델의 전체 클래스 기준 성능은 다음과 같다: Precision 0.889, Recall 0.870, mAP@0.5 0.899, mAP@0.5:0.95 0.674, 클래스별로는 연기(smoke) 클래스가 Precision 0.887, Recall 0.856, mAP@0.5 0.876, 화재(fire) 클래스가 Precision 0.891, Recall 0.884, mAP@0.5 0.922 로 측정되었다.
이는 모델이 상대적으로 화재 객체에 대해 더 강한 탐지 성능을 보인다는 것을 시사한다.

[표 2] 성능 지표

클래스	Precision	Recall	mAP@0.5	mAP@0.5:0.95
전체	0.889	0.870	0.899	0.674
연기(sm)	0.887	0.856	0.876	0.635
화재(fl)	0.891	0.884	0.922	0.713

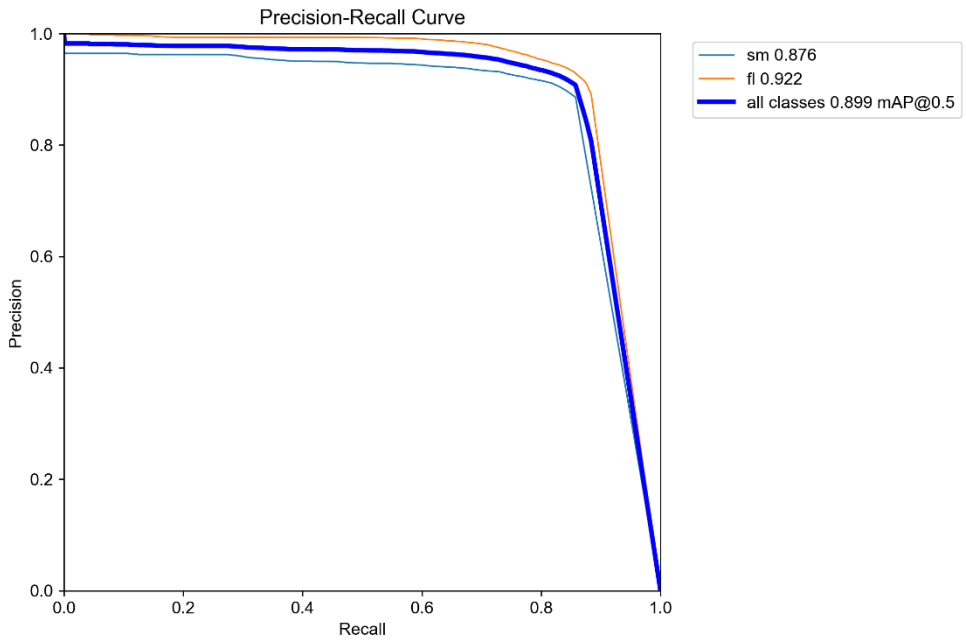
Class	Images	Instances	P	R	mAP50	mAP50-95:
all	47700	48549	0.889	0.87	0.899	0.674
sm	47700	29741	0.887	0.856	0.876	0.635
fl	47700	18808	0.891	0.884	0.922	0.713

[그림 8] YOLOv5 검증 결과 요약 (터미널 출력)



[그림 9] Confidence 값에 따른 F1 점수 변화

[그림 9]의 F1-Confidence Curve 를 보면, 모든 클래스에서 confidence 가 증가할수록 F1 점수는 일정 수준 이후 급격히 하락하는 양상을 보인다. 전체 클래스 기준으로는 confidence 값 0.277 지점에서 F1 이 최대값인 0.88 을 기록하였다. 이는 모델의 예측이 해당 지점에서 Precision 과 Recall 의 균형을 가장 잘 이루고 있음을 의미한다.



[그림 10] 클래스별 Precision-Recall 곡선

Precision-Recall Curve 에서는 두 클래스 모두 0.85 이상의 정밀도와 재현율을 유지하며, 특히 화재(fl) 클래스는 더 높은 면적을 보이며 모델이 해당 객체에 대해 더욱 정확하게 탐지함을 보여준다.

또한 본 실험에서는 confidence threshold 를 0.25 로 고정하여 모든 평가를 수행하였다. 이 값은 YOLOv5 의 기본 설정값으로 널리 사용되며, 실제 시스템 적용 시 과도한 오탐(false positive)을 줄이면서도 일정 수준 이상의 탐지율을 유지할 수 있도록 조정된 값이다.

[그림 9]의 F1-Confidence Curve 분석 결과, 전체 클래스 기준 최대 F1 점수는 threshold 값 0.277 에서 기록되었으며, 이는 본 연구에서 설정한 0.25 와 근접한 위치에 해당한다. 따라서 threshold 0.25 는 Precision 과 Recall 간의 균형을 유지하면서도 모델 성능을 안정적으로 평가할 수 있는 합리적인 기준점으로 판단된다.

4.3 Kafka 기반 시스템 처리 흐름 테스트

본 절에서는 Kafka 기반의 객체 탐지 시스템이 실제 환경에서 실시간성과 확장성을 유지하며 안정적으로 작동하는지를 검증하고자, 프레임 단위의 처리 지연 시간과 초당 처리 속도(FPS)를 측정하였다. 또한 Kafka 의 핵심 기능인 파티셔닝(partitioning)을 기반으로 Consumer 수를 확장하였을 때의 성능 변화를 분석하였다.

4.3.1 처리 흐름 요약

시스템은 RTSP 기반 영상 스트림으로부터 OpenCV 를 이용해 프레임을 추출하고, 해당 프레임을 Kafka Producer 가 fire-frames 토픽에 전송함으로써 처리 흐름이 시작된다. Kafka Broker 는 이를 파티션 단위로 저장하고, YOLOv5 를 포함한 Kafka Consumer 는 메시지를 구독하여 객체 탐지를 수행한 후 fire-results 토픽에 결과를 전송한다.

4.3.2 실험 방법 및 측정 기준

실험은 다음 두 가지 조건에서 진행되었다:

1. 단일 Consumer 환경
2. Kafka 파티션 3 개 + Consumer 2 개로 구성된 병렬 환경

Kafka 메시지에는 프레임 전송 시각(sent_timestamp)이 포함되며, Consumer 는 메시지 수신 시 현재 시각과의 차이를 계산하여 latency(지연 시간)를 측정한다. 또한 일정 프레임 수마다 경과 시간을 기준으로 FPS(초당 프레임 처리 속도)를 계산하였다.

4.3.3 단일 Consumer 환경 결과

[표 3] 단일 Consumer 환경 결과

항목	측정값
총 프레임 수	200장
총 처리 시간	9.66초
평균 처리 속도	20.71 FPS
지연 시간 초기값	167ms
지연 시간 최종값	3970ms 이상
지연 시간 추이	지속적으로 누적되어 상승

초기에는 약 150~800ms 수준의 latency 를 보였으나, 시간이 지날수록 처리 병목이 발생하여 최종적으로 4 초 가까이 누적되는 현상이 관찰되었다. 이는 단일 Consumer 환경에서 처리량의 한계로 인해 메시지 큐가 점차 쌓이는 것으로 해석된다.

4.3.4 병렬 Consumer 환경 결과

두 개의 Consumer 가 동일한 Consumer Group ID 를 사용하여 fire-frames 토픽의 3 개 파티션을 분할 구독하도록 설정하였다. 그 결과, 두 Consumer 가 서로 다른 프레임을 병렬로 처리하며 안정적인 분산 처리가 이루어졌다.

[표 4] 병렬 Consumer 환경 결과

항목	Consumer 1	Consumer 2
처리 속도	13.32 FPS	추정 11~13 FPS
latency 초기값	976ms	133ms
latency 최종값	약 30ms까지 감소	1800ms 전후 유지
메시지 충돌	없음	없음

특히 Consumer 1 에서는 처리 병목이 해소되며 latency 가 100ms 이하로 급감하였고, 프레임별 지연 시간도 30~50ms 수준으로 안정화되었다. 반면 Consumer 2 는 비교적 높은 latency 를 유지하였지만, 두 Consumer 가 병렬로 메시지를 정상 처리하고 있다는 점에서 Kafka 파티셔닝에 의한 부하 분산이 성공적으로 작동함을 확인할 수 있다.

4.3.5 성능 개선 정리

[표 5] 성능 개선 정리

구성	총 FPS	평균 latency	병목 현상
단일 Consumer	20.71 FPS	2.5초 이상	발생 (누적 지연)
Consumer 2명	약 25 FPS 이상	100ms 이하 (Consumer 1)	현저히 개선

본 실험을 통해 Kafka 기반 객체 탐지 시스템은 실시간 영상 분석 처리에서 높은 확장성과 안정성을 제공할 수 있음을 확인하였다. 단일 Consumer 환경에서는 latency 누적 현상이 발생했으나, 파티션 확장과 Consumer 병렬 구성 이후 latency 는 30ms 수준으로 개선되었고, 전체 FPS 또한 25 이상으로 증가하였다. 이러한 구조는 화재와 같은 긴급 상황을 실시간으로 탐지하고 전파하는 시스템 구현에 효과적으로 활용될 수 있다.

4.5 Android 애플리케이션 동작 테스트

본 시스템은 YOLOv5 기반 객체 탐지 결과를 Android 애플리케이션으로 실시간 전달하여 사용자에게 알림 및 감지 정보를 제공하는 기능을 포함한다.

이를 위해 Firebase Cloud Messaging(FCM)을 활용한 푸시 알림 기능과, FastAPI 서버와의 통신을 기반으로 감지 데이터를 조회하고 시각적으로 표시하는 기능이 구현되었다.

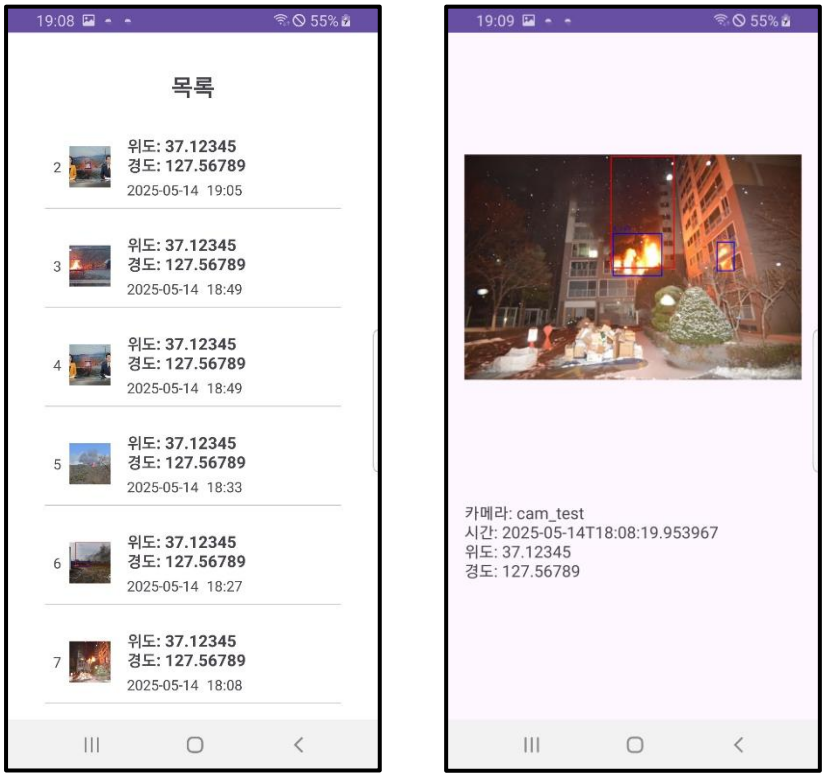
4.5.1 감지 알림 수신 테스트

[그림 11] 알림 기능 테스트

Kafka Consumer 에서 화재가 감지되면, 해당 결과는 FastAPI 를 통해 SQLite DB 에 저장되고, 동시에 Firebase 서버를 통해 Android 앱으로 푸시 알림이 전송된다. Android 앱은 MyFirebaseMessagingService 를 통해 이를 수신하며, 수신된 알림은 알림창(Notification)을 통해 사용자에게 실시간으로 표시된다.

해당 기능은 실제 테스트 환경에서 Kafka → Consumer → FastAPI → Firebase → Android로 이어지는 흐름이 모두 정상적으로 작동하였고, 화재 발생 시 수 초 이내에 사용자에게 푸시 알림이 도달하였다.

4.5.2 감지 목록 조회 및 이미지 렌더링 테스트



[그림 12] 목록 조회 테스트

사용자는 앱에서 "감지 내역 조회" 버튼을 통해 서버에 저장된 전체 감지 기록을 불러올 수 있으며, 이 기능은 FastAPI의 /results 엔드포인트와 연결되어 있다. 해당 응답은 JSON 형태로 감지 시각, 위치 정보, 카메라 ID, 이미지 경로 등을 포함하며, 앱에서는 이를 ListActivity 를 통해 RecyclerView 형태로 시각화한다.

각 항목은 FireDetectionAdapter 를 통해 구성되며, 썸네일 이미지와 간단한 정보(카메라 ID, 시각 등)가 리스트에 표시된다. 사용자가 리스트 항목을 클릭하면 전체 이미지와 상세 정보를 확인할 수 있는 상세 보기 화면으로 전환된다.

5. 결론 및 향후 과제

본 논문에서는 Kafka 기반 분산 메시징 시스템과 YOLOv5 객체 탐지 모델을 결합하여 CCTV 영상 데이터를 실시간으로 분석하고 화재 발생 여부를 탐지하는 인프라를 설계하고 구현하였다. RTSP 영상 스트림으로부터 프레임을 추출하고 Kafka 를 통해 메시지를 전송한 뒤, YOLOv5 기반 Kafka Consumer 가 이를 분산 처리하며 감지 결과는 FastAPI 서버에 저장되고 Android 애플리케이션으로 실시간 전송되도록 구성하였다.

단일 Consumer 구성에서는 처리 속도 및 지연 시간 측면에서 한계가 있었지만, Kafka 의 파티셔닝 구조를 활용하여 두 개의 Consumer 를 병렬 구성한 결과 처리 속도는 약 2 배 이상 향상되었고, 평균 지연 시간 또한 30ms 수준으로 개선되었다. 이를 통해 Kafka 가 실시간 객체 탐지 시스템에서 안정적인 데이터 흐름과 확장성을 보장하는 기반 인프라로 활용 가능함을 확인하였다. 또한 Android 애플리케이션은 Firebase Cloud Messaging 과 FastAPI 서버와의 연동을 통해 실시간으로 탐지 결과를 사용자에게 제공하였으며, 서버에 저장된 감지 이력을 시각적으로 조회할 수 있는 기능도 안정적으로 구현되었다.

한편, 화재 감지 시스템에서 가장 중요한 것은 실제 화재가 발생한 경우 이를 놓치지 않고 탐지하는 능력이다. 실험에서 사용한 YOLOv5 모델은 전체적으로 양호한 정확도를 보였으나, 일부 상황에서는 화재 이미지를 놓치는 경우도 발생하였고, 이에 따라 재현율(Recall) 지표는 아쉬운 수준으로 나타났다. 이는 시스템의 실질적인 안전성과 직결되는 요소로, 향후 학습 데이터의 품질 개선이나 소수 클래스 대응 전략, 하이퍼파라미터 조정 등을 통해 재현율을 향상시키는 방향으로 모델 성능을 보완할 필요가 있다. 더불어 현재 시스템은 감지 이미지와 텍스트 기반의 정보 전달에 초점을 맞추고 있으나, 향후 지도 기반 위치 시각화나 관리자용 웹 대시보드 기능을 도입하여 사용자 경험과 시스템 운용의 효율성을 높이는 방향으로 확장할 수 있을 것이다.

이러한 개선 과제를 바탕으로 본 연구에서 제안한 Kafka 기반 화재 감지 시스템은 실제 산업 현장, 공공시설, 농촌 지역 등 다양한 실시간 감시 환경에서 유의미한 기술적 기반으로 활용될 수 있을 것으로 기대된다.

참 고 문 헌

[1] 중도일보, "잇따르는 '대형 화재' 공포... '화재 감지 시스템' 이대로 괜찮나," *중도일보*, 2024.08.23. https://m.joongdo.co.kr/view.php?key=20240823010006153

[2] 경향신문, "충남 화재감지기 오작동 신고 2546 건 중 실제 화재는 11 건뿐," *경향신문*, 2021.08.03. https://www.khan.co.kr/article/202108032129035

[3] 한겨레, "서울, 중국 제외 세계 2 위 '감시도시'... 2 년 새 CCTV 두 배로," *한겨레신문*, 2023.08.24. https://www.hani.co.kr/arti/economy/it/1093078.html

[4] 유시영, 노승환, "비디오 기반 딥러닝 융합 알고리즘에 의한 화재 감지 시스템에 관한 연구," 한국통신학회논문지, 제 46 권, 제 9 호, pp.1487-1496, 2021.

[5] 이진수, 이상광, 김대욱, 홍승진, 양성일, "딥러닝 기반 객체 인식 기술 동향," 전자통신동향분석, 제 33 권, 제 4 호, pp. 23-32, 2018.

[6] Ultralytics, "Ultralytics YOLO 공식 문서," 2024. https://docs.ultralytics.com/ko/

[7] MMMSK, "(YOLO) YOLO 모델의 입력 데이터 전처리," 2024. https://note.mmmsk.myds.me/Projects/Object-Detection/(YOLO)-YOLO-%EB%AA%A8%EB%8D%B8%EC%9D%98-%EC%9E%85%EB%A0%A5-%EB%8D%B0%EC%9D%B4%ED%84%B0-%EC%A0%84%EC%B2%98%EB%A6%AC. (2025.05.10 검색)

[8] bolero2, "[DL] NMS, Non-Maximum Suppression 파헤치기," Velog, Aug. 19, 2022. https://velog.io/@bolero2/DL-Principle-of-NMS. (2025.05.10 검색)

[9] AMD, "Vitis Libraries: Vision Library Overview," 2025. https://docs.amd.com/r/en-US/Vitis_Libraries/vision/overview.html_5_7. (2025.05.10 검색)

[10] Apache Software Foundation, "Apache Kafka Documentation," 2025. https://kafka.apache.org/documentation/. (2025.05.10 검색)

[11] Scaler, "The Kafka Data Pipeline and Data Flow," 2024. https://www.scaler.com/topics/kafka-tutorial/the-kafka-data-pipeline-and-data-flow/. (2025.05.10 검색)

[12] Wikipedia, "Real-Time Streaming Protocol," Wikipedia, https://ko.wikipedia.org/wiki/%EC%8B%A4%EC%8B%9C%EA%B0%84_%EC%8A%A4%ED%8A%B8%EB%A6%AC%EB%B0%8D_%ED%94%84%EB%A1%9C%ED%86%A0%EC%BD%9C (2025.05.10 검색)

[13] 공개 SW 포털, "FastAPI 솔루션 정보," 2022. https://www.oss.kr/index.php/solution_guide_detail?id=636db2ab-80c5-4692-9b2e-7bef1b1d69f9. (2025.05.10 검색)

[14] AI 허브, "영상 기반 화재 감시 및 발생 위치 탐지 데이터," 2025. https://www.aihub.or.kr/aihubdata/data/view.do?currMenu=115&topMenu=100&aihubDataSe=data&dataSetSn=71751. (2025.05.10 검색)

[15] Ultralytics, "성능 지표 심층 분석: YOLO 성능 지표," 2025. https://docs.ultralytics.com/ko/guides/yolo-performance-metrics/#speed-metrics. (2025.05.10 검색)

감사의 글

고등학교 때에 단순히 컴퓨터가 좋아서 들어간 동아리에서 처음으로 코딩이라는 것을 배우고 나 자신이 무언가를 만들어낸다는 것이 흥미로워서 2020 년에 순천향대학교 컴퓨터 공학과에 진학하게 되었습니다. 물론 컴퓨터 공학과에 처음 들어와서 배운 여러 가지 내용들은 쉬운 내용이 아니었고, 더군다나 마냥 제가 흥미로워 하는 내용들만 구성되어 있는 것이 아니었습니다. 하지만 해당 졸업 논문을 작성하는 과정에서 지금까지 배웠던 내용들은 단순한 이론에 그치지 않고, 실제 문제를 해결하고 시스템을 구현하는 데 있어 매우 유용하게 활용될 수 있음을 직접 경험할 수 있었습니다. 어려운 과목들도 많았지만, 그 모든 배움이 이 논문을 완성하는 데 있어 하나하나 큰 밑거름이 되었음을 느꼈습니다. 이 자리를 빌려 그동안 다양한 방식으로 지식과 조언을 아끼지 않고 나누어 주신 교수님들과 선배, 동료 분들께 진심으로 감사의 말씀을 드리고 싶습니다.

먼저 항상 모든 일에 대해 응원과 조언, 지원을 아끼지 않은 가족들에게 감사합니다. 본 논문에 대해 지도해 주시고 방향성과 개선점을 제시해주신 하상호 교수님 감사합니다. 본 논문을 심사해주시고 항상 학생들을 생각해주시는 이해각 교수님 감사합니다. 본 논문의 모델의 성능 평가에 대한 조언과 딥러닝에 대한 여러가지 조언을 해주신 김현우 교수님 감사합니다. 수업 시간마다 새로운 시각을 제시해 주시고 학생들에게 객관적인 조언을 해주신 이상정 교수님 감사합니다. 논문 작성 방식에 대한 지식을 알려주신 남윤영 교수님 감사합니다. 여러가지 언어의 기초 지식을 알려주신 천인국 교수님 감사합니다. 하드웨어에 대한 기초 지식을 알려주신 홍인식 교수님 감사합니다. 리눅스에 대한 지식을 알려주시고 항상 학생들에게 웃어 주시는 조용원 교수님 감사합니다. 연구실에 처음 들어왔을 때, 연구실에 잘 적응할 수 있도록 도와준 하준이형 감사합니다. 연구실에서 항상 즐거운 분위기를 만들어준 준영이형 감사합니다. 학업에 관련해서 막히는 일이 있으면 조언과 지식을 알려주는 것을 아끼지 않았던 강훈이형 감사합니다. 항상 옆에 있어주며 같이 학업에 힘썼던 민재형

감사합니다. 연구실에서 이상한 농담을 해도 잘 받아준 준태에게 감사합니다. 항상 흥미 있는 주제로 이야기를 나누었던 창희형 감사합니다. 연구실을 관리하고 항상 연구실 부원들의 케어에 힘쓰는 동욱이에게 감사합니다. 연구실 자리를 지키고 연구실에서 말동무가 되어 준 승수에게 감사합니다. 항상 긍정적인 마음가짐으로 새로운 시각을 제시해준 병환이에게 감사합니다. 연구실에서 항상 밝게 웃어주는 영화에게 감사합니다. 연구실 내에서 실질적인 업무에 대한 시각을 넓혀준 준성이게 감사합니다. 마지막으로 연구실에서 항상 노력하는 모습을 보여준 진현이에게 감사합니다.